

Stratum V1, V2, and DATUM

Agenda

- Stratum V1 jobs, shares, and template control

Agenda

- Stratum V1 jobs, shares, and template control
- DATUM's gateway architecture

Agenda

- Stratum V1 jobs, shares, and template control
- DATUM's gateway architecture
- Stratum V2's three protocols

Agenda

- Stratum V1 jobs, shares, and template control
- DATUM's gateway architecture
- Stratum V2's three protocols
- FPPS with miner-created templates

Agenda

- Stratum V1 jobs, shares, and template control
- DATUM's gateway architecture
- Stratum V2's three protocols
- FPPS with miner-created templates
- Technical and specification differences

Stratum V1

Terms

- **Block template:** candidate block contents before proof of work

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** search space derived from a block template

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** search space derived from a block template
- **Network target:** proof-of-work threshold for a valid Bitcoin block

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** search space derived from a block template
- **Network target:** proof-of-work threshold for a valid Bitcoin block
- **Share target:** easier threshold used by a pool to measure work

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** search space derived from a block template
- **Network target:** proof-of-work threshold for a valid Bitcoin block
- **Share target:** easier threshold used by a pool to measure work
- **Share:** proof of work valid for the pool, usually not for the Bitcoin network

Terms

- **Block template:** candidate block contents before proof of work
- **Mining job:** search space derived from a block template
- **Network target:** proof-of-work threshold for a valid Bitcoin block
- **Share target:** easier threshold used by a pool to measure work
- **Share:** proof of work valid for the pool, usually not for the Bitcoin network
- **Coinbase transaction:** first transaction; claims subsidy and fees and carries the extranonce

Stratum V1



jobs flow right, shares flow left

V1 job flow

- The pool builds a block template and derives a mining job

V1 job flow

- The pool builds a block template and derives a mining job
- `mining.notify` sends the previous block hash, coinbase parts, Merkle branches, version, `nBits`, and time

V1 job flow

- The pool builds a block template and derives a mining job
- `mining.notify` sends the previous block hash, coinbase parts, Merkle branches, version, nBits, and time
- Extranonce parameters and the share target are supplied separately

V1 job flow

- The pool builds a block template and derives a mining job
- `mining.notify` sends the previous block hash, coinbase parts, Merkle branches, version, nBits, and time
- Extranonce parameters and the share target are supplied separately
- The miner constructs the coinbase and Merkle root, then searches nonce, extranonce2, time, and allowed version bits

V1 job flow

- The pool builds a block template and derives a mining job
- `mining.notify` sends the previous block hash, coinbase parts, Merkle branches, version, nBits, and time
- Extranonce parameters and the share target are supplied separately
- The miner constructs the coinbase and Merkle root, then searches nonce, extranonce2, time, and allowed version bits
- The pool validates submitted shares and accounts for the measured work

Miner-controlled fields

- Extranonce2 changes the coinbase transaction

Miner-controlled fields

- Extranonce2 changes the coinbase transaction
- The new coinbase transaction changes the Merkle root

Miner-controlled fields

- Extranonce2 changes the coinbase transaction
- The new coinbase transaction changes the Merkle root
- Nonce, time, and permitted version bits provide additional header search space

Miner-controlled fields

- Extranonce2 changes the coinbase transaction
- The new coinbase transaction changes the Merkle root
- Nonce, time, and permitted version bits provide additional header search space
- These fields produce many candidate headers from one pool job

Miner-controlled fields

- Extranonce2 changes the coinbase transaction
- The new coinbase transaction changes the Merkle root
- Nonce, time, and permitted version bits provide additional header search space
- These fields produce many candidate headers from one pool job
- They do not change the non-coinbase transaction set

Pool controls the template

- ASICs do not maintain mempools

Pool controls the template

- ASICs do not maintain mempools
- The pool's node and template builder select the transactions

Pool controls the template

- ASICs do not maintain mempools
- The pool's node and template builder select the transactions
- A mining proxy can distribute a job, but cannot replace its transaction set

Pool controls the template

- ASICs do not maintain mempools
- The pool's node and template builder select the transactions
- A mining proxy can distribute a job, but cannot replace its transaction set
- A miner can reject the job or change pools, but cannot modify the current template

Pool controls the template

- ASICs do not maintain mempools
- The pool's node and template builder select the transactions
- A mining proxy can distribute a job, but cannot replace its transaction set
- A miner can reject the job or change pools, but cannot modify the current template
- Pool operators therefore select block contents for their connected hash rate

V1 limitations

- **Pool-built templates**
 - transaction selection is concentrated at pools

V1 limitations

- **Pool-built templates**
 - transaction selection is concentrated at pools
- **JSON wire format**
 - more bytes and parsing than a binary protocol

V1 limitations

- **Pool-built templates**
 - transaction selection is concentrated at pools
- **JSON wire format**
 - more bytes and parsing than a binary protocol
- **Informal specification, dialects, and extensions**
 - compatibility depends on established implementation behavior

V1 limitations

- **Pool-built templates**
 - transaction selection is concentrated at pools
- **JSON wire format**
 - more bytes and parsing than a binary protocol
- **Informal specification, dialects, and extensions**
 - compatibility depends on established implementation behavior
- **No native authentication or encryption**

V1 limitations

- **Pool-built templates**
 - transaction selection is concentrated at pools
- **JSON wire format**
 - more bytes and parsing than a binary protocol
- **Informal specification, dialects, and extensions**
 - compatibility depends on established implementation behavior
- **No native authentication or encryption**
- **No standard channel model for proxy aggregation**

V1 limitations

- **Pool-built templates**
 - transaction selection is concentrated at pools
- **JSON wire format**
 - more bytes and parsing than a binary protocol
- **Informal specification, dialects, and extensions**
 - compatibility depends on established implementation behavior
- **No native authentication or encryption**
- **No standard channel model for proxy aggregation**
- **Coinbase and Merkle processing below the pool**
 - additional bandwidth and firmware or proxy work

Template Control

Split construction from accounting

Template construction

Transactions in the candidate block

Pool accounting

Accepted shares and miner payouts

DATUM and Stratum V2 can move template construction to the miner while retaining pool accounting.

Miner-created templates

- The miner's Bitcoin node supplies the template

Miner-created templates

- The miner's Bitcoin node supplies the template
- Its mempool and policy determine the transaction set

Miner-created templates

- The miner's Bitcoin node supplies the template
- Its mempool and policy determine the transaction set
- The pool supplies required coinbase outputs and validates shares

Miner-created templates

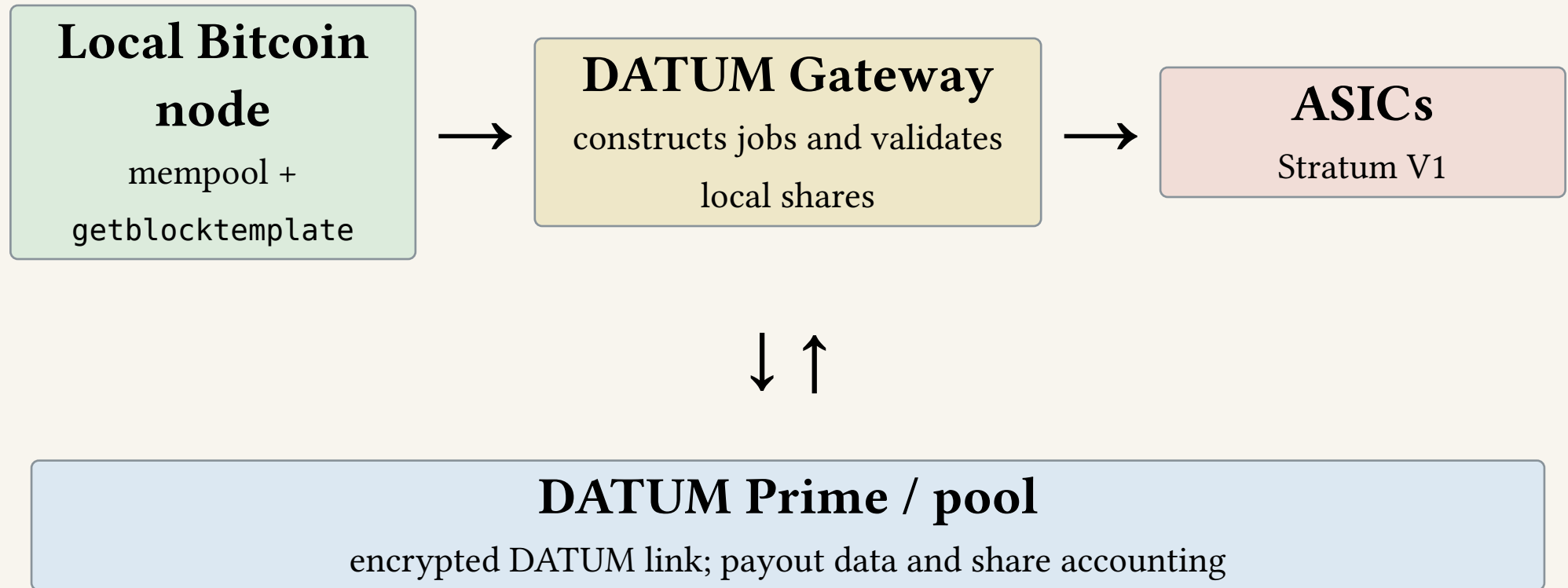
- The miner's Bitcoin node supplies the template
- Its mempool and policy determine the transaction set
- The pool supplies required coinbase outputs and validates shares
- The miner's node can submit a solved block

Miner-created templates

- The miner's Bitcoin node supplies the template
- Its mempool and policy determine the transaction set
- The pool supplies required coinbase outputs and validates shares
- The miner's node can submit a solved block
- FPPS additionally requires a credible transaction-fee value for each declared job

DATUM

DATUM architecture



DATUM custom-template flow

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local Bitcoin node builds a block template
- 2.
- 3.
- 4.
- 5.
- 6.

DATUM custom-template flow

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local Bitcoin node builds a block template
2. The pool provides its payout requirements
- 3.
- 4.
- 5.
- 6.

DATUM custom-template flow

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local Bitcoin node builds a block template
2. The pool provides its payout requirements
3. The Gateway combines both into a Stratum V1 mining job
- 4.
- 5.
- 6.

DATUM custom-template flow

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local Bitcoin node builds a block template
2. The pool provides its payout requirements
3. The Gateway combines both into a Stratum V1 mining job
4. ASICs mine the job and return shares through the Gateway
- 5.
- 6.

DATUM custom-template flow

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local Bitcoin node builds a block template
2. The pool provides its payout requirements
3. The Gateway combines both into a Stratum V1 mining job
4. ASICs mine the job and return shares through the Gateway
5. The pool accounts for the accepted shares
- 6.

DATUM custom-template flow

DATUM: Decentralized Alternative Templates for Universal Mining

1. The local Bitcoin node builds a block template
2. The pool provides its payout requirements
3. The Gateway combines both into a Stratum V1 mining job
4. ASICs mine the job and return shares through the Gateway
5. The pool accounts for the accepted shares
6. The local Bitcoin node submits a solved block

DATUM template policy

- The Gateway creates work only from the miner's local node

DATUM template policy

- The Gateway creates work only from the miner's local node
- DATUM defines no pool-supplied template path

DATUM template policy

- The Gateway creates work only from the miner's local node
- DATUM defines no pool-supplied template path
- Local template construction is mandatory

DATUM template policy

- The Gateway creates work only from the miner's local node
- DATUM defines no pool-supplied template path
- Local template construction is mandatory
- Existing ASICs continue to use Stratum V1

DATUM template policy

- The Gateway creates work only from the miner's local node
- DATUM defines no pool-supplied template path
- Local template construction is mandatory
- Existing ASICs continue to use Stratum V1
- The node-facing interface remains `getblocktemplate`

DATUM template policy

- The Gateway creates work only from the miner's local node
- DATUM defines no pool-supplied template path
- Local template construction is mandatory
- Existing ASICs continue to use Stratum V1
- The node-facing interface remains getblocktemplate
- The Gateway-to-pool protocol is DATUM-specific and described as evolving

Stratum V2

Three protocols

Protocol	Responsibility
Mining Protocol	Jobs, channels, targets, shares, and solutions
Template Distribution Protocol	Templates and transaction data from a full node
Job Declaration Protocol	Miner-created jobs proposed to a pool

The Mining Protocol can run alone. Job Declaration is optional, so SV2 does not by itself move template construction to miners.

SV2 roles

- **Template Provider:** node-side source of block templates

SV2 roles

- **Template Provider:** node-side source of block templates
- **Job Declarator Client (JDC):** miner-side coordinator for custom jobs

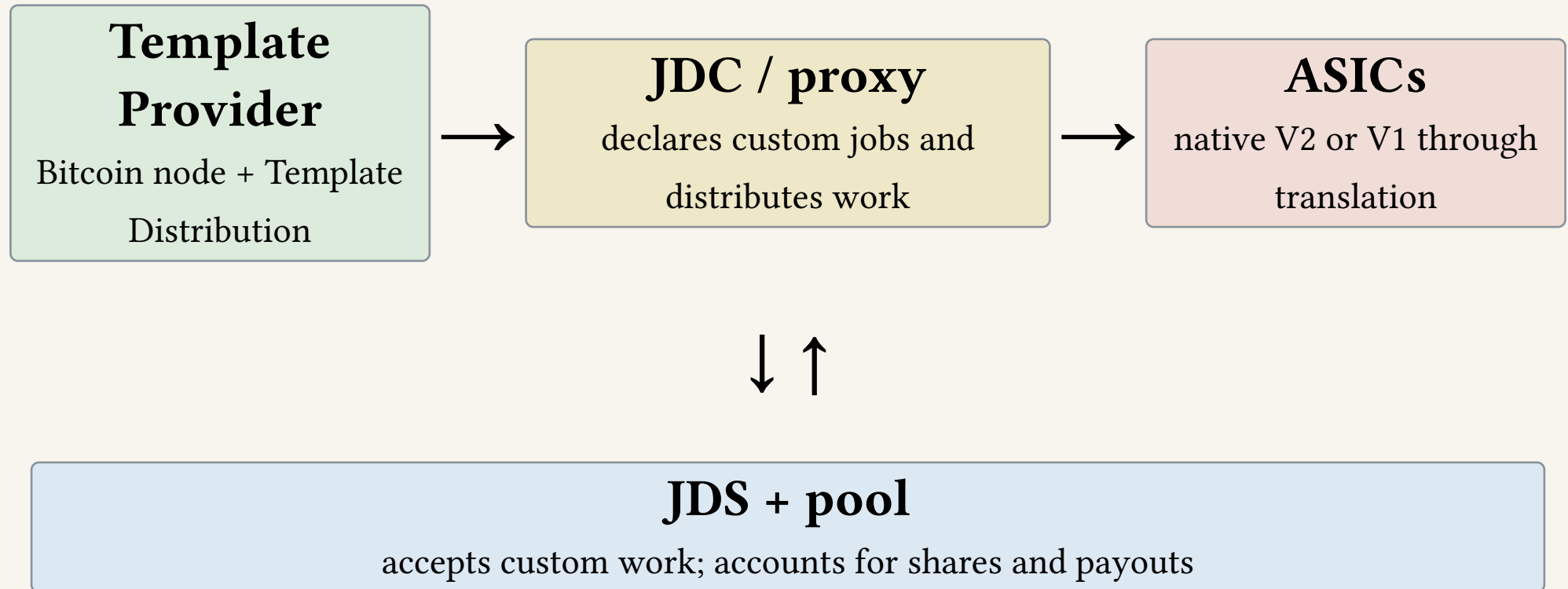
SV2 roles

- **Template Provider:** node-side source of block templates
- **Job Declarator Client (JDC):** miner-side coordinator for custom jobs
- **Job Declarator Server (JDS):** pool-side job validation and acceptance

SV2 roles

- **Template Provider:** node-side source of block templates
- **Job Declarator Client (JDC):** miner-side coordinator for custom jobs
- **Job Declarator Server (JDS):** pool-side job validation and acceptance
- **Mining proxy:** distributes jobs and aggregates downstream channels

Stratum V2 architecture



Stratum V2 custom-template flow

1. The miner's Bitcoin node builds a block template
- 2.
- 3.
- 4.
- 5.
- 6.

Stratum V2 custom-template flow

1. The miner's Bitcoin node builds a block template
2. The pool provides its payout requirements
- 3.
- 4.
- 5.
- 6.

Stratum V2 custom-template flow

1. The miner's Bitcoin node builds a block template
2. The pool provides its payout requirements
3. The miner-side JDC combines both into a custom job
- 4.
- 5.
- 6.

Stratum V2 custom-template flow

1. The miner's Bitcoin node builds a block template
2. The pool provides its payout requirements
3. The miner-side JDC combines both into a custom job
4. The JDC declares the job to the pool-side JDS
- 5.
- 6.

Stratum V2 custom-template flow

1. The miner's Bitcoin node builds a block template
2. The pool provides its payout requirements
3. The miner-side JDC combines both into a custom job
4. The JDC declares the job to the pool-side JDS
5. The pool validates and accepts the job
- 6.

Stratum V2 custom-template flow

1. The miner's Bitcoin node builds a block template
2. The pool provides its payout requirements
3. The miner-side JDC combines both into a custom job
4. The JDC declares the job to the pool-side JDS
5. The pool validates and accepts the job
6. Mining devices receive the job; shares go to the pool and a solved block is submitted locally

Two Job Declaration modes

Property	Full-template	Coinbase-only
Transaction set	Declared; missing transactions can be requested	Not revealed to pool or JDS
Fee and validity checks	JDS can verify the proposed template	Pool cannot independently verify the transaction set
Privacy	JDS learns the template	Transaction set remains private
Pool risk	Validated job	Requires a separate risk policy

SV2 transport and work distribution

- Binary framing with a published specification

SV2 transport and work distribution

- Binary framing with a published specification
- Noise handshake and AEAD transport

SV2 transport and work distribution

- Binary framing with a published specification
- Noise handshake and AEAD transport
- Multiple channels per connection and proxy aggregation

SV2 transport and work distribution

- Binary framing with a published specification
- Noise handshake and AEAD transport
- Multiple channels per connection and proxy aggregation
- Header-only jobs that keep coinbase and Merkle processing away from ASICs

SV2 transport and work distribution

- Binary framing with a published specification
- Noise handshake and AEAD transport
- Multiple channels per connection and proxy aggregation
- Header-only jobs that keep coinbase and Merkle processing away from ASICs
- Future jobs prepared before a new previous-block hash arrives

SV2 transport and work distribution

- Binary framing with a published specification
- Noise handshake and AEAD transport
- Multiple channels per connection and proxy aggregation
- Header-only jobs that keep coinbase and Merkle processing away from ASICs
- Future jobs prepared before a new previous-block hash arrives
- Native version rolling and specified protocol extensions

FPPS

FPPS requirement

FPPS pays the expected subsidy and transaction fees for each accepted share.

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F)$$

- P_{share} : gross payout for one accepted share

FPPS requirement

FPPS pays the expected subsidy and transaction fees for each accepted share.

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F)$$

- P_{share} : gross payout for one accepted share
- D_{eff} : difficulty credited to the share

FPPS requirement

FPPS pays the expected subsidy and transaction fees for each accepted share.

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F)$$

- P_{share} : gross payout for one accepted share
- D_{eff} : difficulty credited to the share
- D_{net} : Bitcoin network difficulty

FPPS requirement

FPPS pays the expected subsidy and transaction fees for each accepted share.

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F)$$

- P_{share} : gross payout for one accepted share
- D_{eff} : difficulty credited to the share
- D_{net} : Bitcoin network difficulty
- R_{base} : block subsidy

FPPS requirement

FPPS pays the expected subsidy and transaction fees for each accepted share.

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F)$$

- P_{share} : gross payout for one accepted share
- D_{eff} : difficulty credited to the share
- D_{net} : Bitcoin network difficulty
- R_{base} : block subsidy
- F : transaction fees assigned to the job

The pool pays before a block is found. It must verify or limit F .

FPPS with full-template declaration

1. The miner selects the transactions
- 2.
- 3.
- 4.
- 5.

FPFS with full-template declaration

1. The miner selects the transactions
2. The JDC declares the transaction set and coinbase data
- 3.
- 4.
- 5.

FPPS with full-template declaration

1. The miner selects the transactions
2. The JDC declares the transaction set and coinbase data
3. The JDS obtains missing transactions and validates the template
- 4.
- 5.

FPPS with full-template declaration

1. The miner selects the transactions
2. The JDC declares the transaction set and coinbase data
3. The JDS obtains missing transactions and validates the template
4. The pool computes F_{verified}
- 5.

FPPS with full-template declaration

1. The miner selects the transactions
2. The JDC declares the transaction set and coinbase data
3. The JDS obtains missing transactions and validates the template
4. The pool computes F_{verified}
5. The accepted job and its shares use the verified fee value

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F_{\text{verified}})$$

No fee cap or additional incentive mechanism is required.

FPPS with coinbase-only declaration

The pool cannot verify the transaction set before a block is found.

$$F_{\text{cap}} = \min(F_{\text{declared}}, \gamma F_{\text{avg}})$$

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F_{\text{cap}})$$

$$B_{\text{surplus}} = \lambda \cdot \max(0, F_{\text{real}} - F_{\text{cap}})$$

- γ (“gamma”) multiplies the recent average fee, for example 1.4

FPPS with coinbase-only declaration

The pool cannot verify the transaction set before a block is found.

$$F_{\text{cap}} = \min(F_{\text{declared}}, \gamma F_{\text{avg}})$$

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F_{\text{cap}})$$

$$B_{\text{surplus}} = \lambda \cdot \max(0, F_{\text{real}} - F_{\text{cap}})$$

- γ (“gamma”) multiplies the recent average fee, for example 1.4
- The fee cap bounds exposure; it does not verify the fee claim

FPPS with coinbase-only declaration

The pool cannot verify the transaction set before a block is found.

$$F_{\text{cap}} = \min(F_{\text{declared}}, \gamma F_{\text{avg}})$$

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F_{\text{cap}})$$

$$B_{\text{surplus}} = \lambda \cdot \max(0, F_{\text{real}} - F_{\text{cap}})$$

- γ (“gamma”) multiplies the recent average fee, for example 1.4
- The fee cap bounds exposure; it does not verify the fee claim
- λ is the fraction of verified surplus paid, from 0 to 1

FPPS with coinbase-only declaration

The pool cannot verify the transaction set before a block is found.

$$F_{\text{cap}} = \min(F_{\text{declared}}, \gamma F_{\text{avg}})$$

$$P_{\text{share}} = \frac{D_{\text{eff}}}{D_{\text{net}}} (R_{\text{base}} + F_{\text{cap}})$$

$$B_{\text{surplus}} = \lambda \cdot \max(0, F_{\text{real}} - F_{\text{cap}})$$

- γ (“gamma”) multiplies the recent average fee, for example 1.4
- The fee cap bounds exposure; it does not verify the fee claim
- λ is the fraction of verified surplus paid, from 0 to 1
- $\max(0, x)$ returns zero when x is negative

Claim that FPPS is impossible with SV2

- The claim is false as a general statement

Claim that FPPS is impossible with SV2

- The claim is false as a general statement
- Full-template Job Declaration lets the miner select transactions and the pool verify their fees

Claim that FPPS is impossible with SV2

- The claim is false as a general statement
- Full-template Job Declaration lets the miner select transactions and the pool verify their fees
- Coinbase-only declaration requires a separate risk policy

Claim that FPPS is impossible with SV2

- The claim is false as a general statement
- Full-template Job Declaration lets the miner select transactions and the pool verify their fees
- Coinbase-only declaration requires a separate risk policy
- SV2 does not define FPPS policy; it supplies the data required to implement it

Based on Pavlenex, *FPPS with JD Modes (Stratum V2 Compatible)*.

Comparison

SV2 and DATUM

Dimension	Stratum V2	DATUM
Scope	Mining protocol suite	Gateway for local templates and pool accounting
ASIC-facing link	Native V2 or translated V1	V1
Node-facing link	Template Distribution Protocol	getblocktemplate RPC
Template source	Pool jobs or miner-created jobs	Miner's local node only
Template validation	Full-template or coinbase-only declaration	Pool-specific behavior
FPPS path	Verified full-template accounting	No interoperable protocol flow specified
Specification	Published modular protocol standard	Open implementation; pool protocol described as evolving
Remote security	Specified Noise/AEAD transport	Encrypted DATUM transport

Why Stratum V2 is preferable

- Replaces V1 on the pool link and can replace it inside the farm

Why Stratum V2 is preferable

- Replaces V1 on the pool link and can replace it inside the farm
- Defines interfaces for nodes, pools, proxies, and mining devices

Why Stratum V2 is preferable

- Replaces V1 on the pool link and can replace it inside the farm
- Defines interfaces for nodes, pools, proxies, and mining devices
- Provides a published protocol standard independent of one implementation

Why Stratum V2 is preferable

- Replaces V1 on the pool link and can replace it inside the farm
- Defines interfaces for nodes, pools, proxies, and mining devices
- Provides a published protocol standard independent of one implementation
- Supports pool-created and miner-created jobs

Why Stratum V2 is preferable

- Replaces V1 on the pool link and can replace it inside the farm
- Defines interfaces for nodes, pools, proxies, and mining devices
- Provides a published protocol standard independent of one implementation
- Supports pool-created and miner-created jobs
- Supports verifiable FPPS through full-template Job Declaration

Why Stratum V2 is preferable

- Replaces V1 on the pool link and can replace it inside the farm
- Defines interfaces for nodes, pools, proxies, and mining devices
- Provides a published protocol standard independent of one implementation
- Supports pool-created and miner-created jobs
- Supports verifiable FPPS through full-template Job Declaration
- Standardizes framing, transport security, channels, and extensions

Why Stratum V2 is preferable

- Replaces V1 on the pool link and can replace it inside the farm
- Defines interfaces for nodes, pools, proxies, and mining devices
- Provides a published protocol standard independent of one implementation
- Supports pool-created and miner-created jobs
- Supports verifiable FPPS through full-template Job Declaration
- Standardizes framing, transport security, channels, and extensions
- Allows V1 translation during migration

DATUM trade-offs

- Existing V1 ASICs work unchanged behind one Gateway

DATUM trade-offs

- Existing V1 ASICs work unchanged behind one Gateway
- Local template construction is mandatory

DATUM trade-offs

- Existing V1 ASICs work unchanged behind one Gateway
- Local template construction is mandatory
- V1 and getblocktemplate remain part of the architecture

DATUM trade-offs

- Existing V1 ASICs work unchanged behind one Gateway
- Local template construction is mandatory
- V1 and getblocktemplate remain part of the architecture
- The pool protocol is specific to DATUM and still evolving

DATUM trade-offs

- Existing V1 ASICs work unchanged behind one Gateway
- Local template construction is mandatory
- V1 and getblocktemplate remain part of the architecture
- The pool protocol is specific to DATUM and still evolving
- FPPS validation is not defined as an interoperable protocol flow

DATUM trade-offs

- Existing V1 ASICs work unchanged behind one Gateway
- Local template construction is mandatory
- V1 and getblocktemplate remain part of the architecture
- The pool protocol is specific to DATUM and still evolving
- FPPS validation is not defined as an interoperable protocol flow
- An open implementation is not equivalent to an open protocol standard

Comparison

Q&A